1、注册界面绘制

```
需求：创建一个界面，用于用户注册信息，包括用户名与密码，并保存用户信息。

要求：用户名与密码不能为空，用户名使用数字注册且长度为11位，密码长度为6位）,清除功能

需要的组件：  一个按钮 QPushButton，两个标签 QLabel，两个编辑框QLineEdit，对话框 QMessageBox

实现方式：手写代码完成开发
```

文件基本操作

```python
# 数据写入到文件中：  with open的方式 打开文件，操作结束之后，文件会自动关闭
with open("user.txt", mode="a+", encoding="utf-8") as f:
    info = "account" + "," + "pwd" + "\n"
    f.write(info)  # write需要的参数是 str字符串
```

2、用到的类、函数

2.1 QLabel

在 PyQt5 中，`QLabel` 是一个非常常用的控件，用于显示文本或图像。它是 GUI 应用程序中最基础的控件之一，通常用于显示静态信息（如标题、说明文字）或简单的图片。虽然 `QLabel` 本身不支持用户交互（即用户不能编辑其内容），但它可以显示富文本（HTML 格式）和超链接。

`QLabel` 的主要作用是显示静态内容，包括：

- **文本**：可以显示纯文本或富文本（HTML 格式）。
- **图像**：可以显示图片。
- **超链接**：可以显示可点击的超链接。

它通常用于以下场景：

华清远见|元宇宙实验中心 yyzlab.com.cn

- 显示窗口中的标题、标签或说明文字。
- 显示图片或图标。
- 显示超链接，允许用户点击跳转到外部网页。

2.1.1 QLabel 的导入方式

```
from PyQt5.QtWidgets import QLabel
```

2.1.2 常用方法

2.1.2.1 设置和获取文本

- `setText(str)`：设置标签上显示的文本。

  ```
  label.setText("Hello, PyQt5!")
  ```

- `text()`：获取标签上显示的文本。

  ```
  text = label.text()
  print(f"Label text: {text}")
  ```

2.1.2.2 设置和获取图片

- `setPixmap(QPixmap)`：为标签设置图片。

  ```
  from PyQt5.QtGui import QPixmap
  label.setPixmap(QPixmap("image.png"))
  ```

2.1.2.3 设置对齐方式

- `setAlignment(Qt.AlignmentFlag)`：设置文本或图片的对齐方式。

  ```
  from PyQt5.QtCore import Qt
  label.setAlignment(Qt.AlignCenter)  # 居中对齐
  ```

2.1.2.4 设置超链接

- `setText(str)`：可以通过 HTML 设置超链接。

  ```
  label.setText('<a href="https://www.example.com">点击访问链接</a>')
  ```

- `setOpenExternalLinks(bool)`：设置是否允许打开外部链接。

  ```
  label.setOpenExternalLinks(True)  # 允许点击链接打开外部浏览器
  ```

2.1.2.5 清空内容

- `clear()`：清空标签的内容。

  ```
  label.clear()
  ```

华清远见|元宇宙实验中心
yyzlab.com.cn

2.1.2.6 设置富文本（HTML 格式）

- `setText(str)`：可以通过 HTML 格式设置富文本内容。

```
label.setText("<h1>这是一个标题</h1><p>这是段落。</p>")
```

2.1.3 典型应用场景

- **显示静态文本**：如窗口标题、说明文字等。
- **显示图片**：可以用作图片展示控件。
- **显示超链接**：可以用作简单的超链接展示。
- **显示富文本**：可以显示带有格式的文本内容，如 HTML 页面、带样式的文本等。

2.2 QLineEdit

在 PyQt5 中，`QLineEdit` 是一个非常常用的控件，用于接收用户的单行文本输入。它是 GUI 应用程序中最常见的输入控件之一，通常用于输入用户名、密码、搜索关键字等。`QLineEdit` 提供了多种功能来控制用户输入的内容，例如设置占位符文本、限制输入内容、回显模式（如密码模式）等。

`QLineEdit` 的主要作用是接收用户的单行文本输入。它可以用于以下场景：

- **接收用户输入**：如用户名、密码、搜索关键字等。
- **动态更新界面**：用户输入后，可以根据输入内容动态更新界面。
- **密码输入**：通过设置回显模式，可以隐藏用户输入的密码。

2.2.1 QLineEdit 的导入方式

```
from PyQt5.QtWidgets import QLineEdit
```

2.2.2 常用方法

2.2.2.1 设置和获取文本

- `setText(str)`：设置文本框中的默认文本。

```
line_edit.setText("默认文本")
```

- `text()`：获取用户输入的文本。

```
user_input = line_edit.text()
print(f"User input: {user_input}")
```

2.2.2.2 设置占位符文本

- `setPlaceholderText(str)`：设置占位符文本（提示用户输入的内容）。

```
line_edit.setPlaceholderText("请输入内容")
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.2.2.3 设置回显模式

- `setEchoMode(QLineEdit.EchoMode)`：设置文本框的回显模式，常用于密码输入。
  - `QLineEdit.Normal`：正常显示输入内容（默认）。
  - `QLineEdit.NoEcho`：不显示任何输入内容。
  - `QLineEdit.Password`：以密文形式显示输入内容。
  - `QLineEdit.PasswordEchoOnEdit`：编辑时显示明文，失去焦点后显示密文。

```
line_edit.setEchoMode(QLineEdit.Password)  # 密码模式
```

2.2.2.4 清空文本框

- `clear()`：清空文本框中的内容。

```
line_edit.clear()
```

2.2.2.5 限制输入内容

- `setValidator(QValidator)`：设置验证器，限制用户输入的内容。可以使用 `QIntValidator`、`QDoubleValidator` 或 `QRegExpValidator` 来限制输入为整数、浮点数或符合正则表达式的字符串。

```
from PyQt5.QtGui import QIntValidator

line_edit.setValidator(QIntValidator(0, 100))  # 只能输入 0 到 100 之间的整数
```

2.2.2.6 设置最大长度

- `setMaxLength(int)`：设置文本框中允许的最大字符数。

```
line_edit.setMaxLength(10)  # 最多输入 10 个字符
```

2.2.3 典型应用场景

- **接收用户输入**：如用户名、密码、搜索关键字等。
- **动态更新界面**：用户输入后，可以根据输入内容动态更新界面。
- **密码输入**：通过设置 `setEchoMode(QLineEdit.Password)` 来隐藏用户输入的密码。
- **限制输入内容**：通过设置验证器，确保用户只能输入符合特定规则的内容（如数字、字母等）。

2.3 QMessageBox

在 PyQt5 中，`QMessageBox` 是一个非常常用的类，用于显示消息框。它通常用于向用户显示信息、警告、错误或询问用户的选择。`QMessageBox` 是一个模态对话框，意味着在用户关闭消息框之前，应用程序的其他部分无法与用户交互。

`QMessageBox` 的主要作用是通过弹出消息框与用户进行交互。它可以用于以下场景：

- **显示信息**：如操作成功、操作失败等提示信息。
- **显示警告**：如用户输入无效、文件不存在等警告信息。
- **显示错误**：如程序运行时发生错误。
- **询问用户选择**：如确认删除、确认退出等操作。

华清远见元宇宙实验中心 yyzlab.com.cn

2.3.1 QMessageBox 的导入方式

```
from PyQt5.QtWidgets import QMessageBox
```

2.3.2 常用方法

2.3.2.1 静态方法

`QMessageBox` 提供了几个静态方法，用于快速显示不同类型的消息框。这些方法会自动创建并显示消息框，并返回用户的选择（如果有按钮的话）。

- `information(parent, title, message)`：显示信息框。

  ```
  QMessageBox.information(None, "信息", "这是一个信息框")
  ```

- `warning(parent, title, message)`：显示警告框。

  ```
  QMessageBox.warning(None, "警告", "这是一个警告框")
  ```

- `critical(parent, title, message)`：显示错误框。

  ```
  QMessageBox.critical(None, "错误", "这是一个错误框")
  ```

- `question(parent, title, message, buttons=QMessageBox.Yes | QMessageBox.No, defaultButton=QMessageBox.No)`：显示询问框，并返回用户的选择。

  ```
  reply = QMessageBox.question(None, "确认", "你确定要继续吗？", QMessageBox.Yes |
  QMessageBox.No, QMessageBox.No)
  if reply == QMessageBox.Yes:
      print("用户选择了 Yes")
  else:
      print("用户选择了 No")
  ```

2.3.2.2 自定义消息框

除了使用静态方法外，你还可以通过创建 `QMessageBox` 对象来自定义消息框的内容和按钮。

- `setText(str)`：设置消息框中显示的主要内容。

  ```
  msg_box = QMessageBox()
  msg_box.setText("这是一个自定义消息框")
  ```

- `setInformativeText(str)`：设置消息框中的详细信息（可选）。

  ```
  msg_box.setInformativeText("这是详细信息")
  ```

- `setDetailedText(str)`：设置消息框中的详细文本（可选），用户可以点击 "Show Details" 按钮查看。

  ```
  msg_box.setDetailedText("这是详细的错误信息")
  ```

仅供学习使用 yyzlab.com.cn

- `setIcon(QMessageBox.Icon)`：设置消息框的图标。
  - `QMessageBox.Information`：信息图标。
  - `QMessageBox.Warning`：警告图标。
  - `QMessageBox.Critical`：错误图标。
  - `QMessageBox.Question`：问题图标。

```
msg_box.setIcon(QMessageBox.Warning)
```

- `setStandardButtons(buttons)`：设置消息框中的标准按钮。

```
msg_box.setStandardButtons(QMessageBox.Ok | QMessageBox.Cancel)
```

- `exec_()`：显示消息框并进入模态对话框模式，直到用户关闭消息框。

```
reply = msg_box.exec_()
if reply == QMessageBox.Ok:
    print("用户点击了 OK")
elif reply == QMessageBox.Cancel:
    print("用户点击了 Cancel")
```

2.3.3 常见按钮

`QMessageBox` 提供了多种标准按钮，常用的有：

- `QMessageBox.Ok`：确认按钮。
- `QMessageBox.Cancel`：取消按钮。
- `QMessageBox.Yes`：是按钮。
- `QMessageBox.No`：否按钮。
- `QMessageBox.Close`：关闭按钮。
- `QMessageBox.Apply`：应用按钮。
- `QMessageBox.Reset`：重置按钮。

你可以通过按位或运算符（`|`）组合多个按钮。例如：

```
msg_box.setStandardButtons(QMessageBox.Yes | QMessageBox.No |
QMessageBox.Cancel)
```

2.3.4 典型应用场景

- **显示信息**：如操作成功、操作失败等提示信息。
- **显示警告**：如用户输入无效、文件不存在等警告信息。
- **显示错误**：如程序运行时发生错误。
- **询问用户选择**：如确认删除、确认退出等操作。

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见|元宇宙实验中心
yyzlab.com.cn